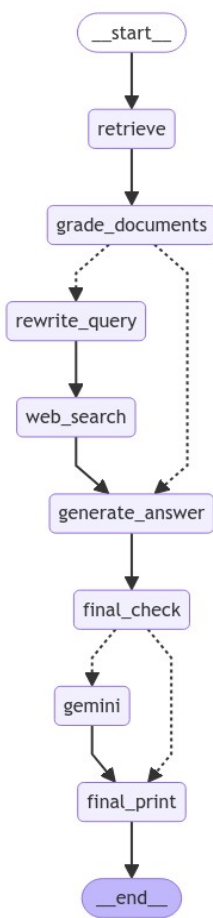


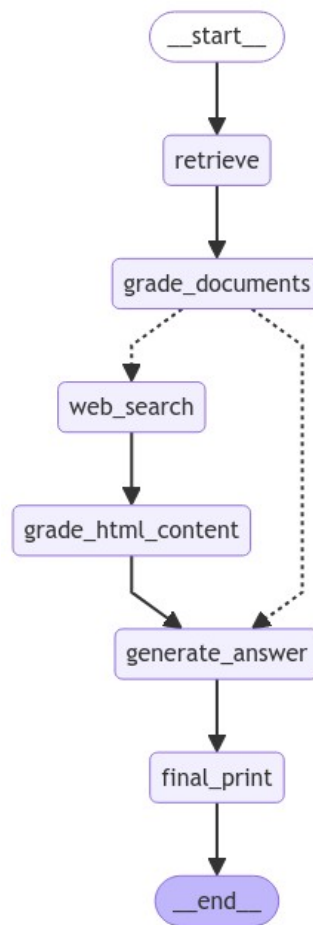
Evolution of my thought process during the assignment

- My first thought after the first reading of the problem statement was to build something along the lines of a question answering system. I thought of maybe trying a simple approach using a transformer model trained on QnA tasks to solve it, but I thought it wouldn't be able to pick out a lot of the information from the data given unless fine-tuned, besides some of the questions required data that wasn't present in what was given.
- I decided to go ahead with using LLMs for the task as they require little to no fine tuning usually and are able to generalize well most of the time.
- Since the task required me to answer questions based on three documents given, I had to connect the LLM to an external data source, which led me to use some form of RAG
- It took me a while to get the chunking of documents right to insert into the database. I tried multiple chunking methods and chunk values to prevent loss of context.
- The local rag system worked fine for a few questions, but it became clear that some of the questions in the template required data that was not present in the given documents.
- I picked up some data from wikipedia, added it to the database manually and queried it to see how it was working, there was an improvement in the number of queries answered. But some questions (like a general job description and company organizational charts which were not publicly available) were still not being answered.
- I tried making calls to the gemini api to see if there would be some improvement in the number of queries answered and it worked well.
- So far the data that I was using was being restricted to just the provided documents, wikipedia and the gemini api. I saw some instances of the gemini api hallucinating, which led me to think that I should try to find other data sources
- As I was suggested, I looked for some search APIs and I found some which worked well. The one I used was Tavily
- There were now four sources of data, the provided documents, gemini api, wikipedia and the data I got from Tavily. The wikipedia data could be returned from Tavily itself. I had a general idea of which data source worked best for which query, but while running a few tests I saw things outside of that general idea happen. This led to the thought process of using agents in some way (this was my first time building agents so I decided to use langraph) so as to let an LLM make the decision of which tool to choose.
- In short, the solution evolved from a simple RAG system to one with agentic involvement.

Architecture for both the questions:



Architecture for Q1



Architecture for Q2

- The architecture is a modified version of the “Corrective RAG” architecture that I saw online
- The “retrieve” node returns relevant documents from the vector database
- The “grade documents” node uses an LLM to check if the retrieved documents are relevant to the query by checking if any keywords from the query are present in the retrieved documents or if there is some match in the semantic meaning.
- The “rewrite query” node uses an LLM to rewrite the query such that it produces good search results
- The “web search” node uses Tavily to generate results for the query
- The “grade html content” uses an LLM to check to see which of the returned search results are relevant to the query.

- The “final check” node uses an LLM to check if the answer generated is relevant to the query (I could have added the same node to the architecture for the second question too, but I didn’t because the costs for using the OpenAI API was going up)
- The “gemini” node uses the g4f package to make calls to LLM APIs.
- I made a small modification to the architecture apart from the ones mentioned. The documents that seemed to be relevant to the query at the “grade html documents” node were pushed into the database which was not done in the first architecture.
 - The reason behind this is: I noticed that the web results returned after querying the web for different queries in the second question were almost the same. They were usually the about/membership pages or the linkedin pages of the organizations that granted the fellowships, so instead of making the same API calls so many times, it seemed better to push the results into a database where the retrieval was faster than an API call

Reason for difference in the architectures:

- After running a few tests I noticed that the LLM rewriting the query didn’t work as well for the queries relating to the second question, I was trying to pass some context about the organization which offers memberships in some cases (which was not required for the first question) like the YEH membership (YEH was being recognized on the web as something else unless the context of “It’s a Global Founder-Focused Membership Community, based in Silicon Valley.” was passed in) and that was getting missed by the LLM while rephrasing.
- The documents generated returned were unrelated to the query sometimes (happened with the YEH membership again), so I decided to add a document grader node again, before passing the returned documents to the “generate answer” node to filter out some unnecessary documents if they were returned from the web search.
- I did not use the gemini api in the second architecture as I did not find a use case for it. It was useful for the first question as it helped generate the job descriptions and the organizational charts which were usually not available online.
- I didn’t have to grade the documents retrieved from the web in the first architecture as it was being returned correctly from the tests I ran as the queries usually involved well known companies. I saw some instances of wrong search results being returned for YEH which led me to add the grader node for the search results.

Some extra things

- The architecture mentions that I used the gemini api for answer generation at the end. I initially used the gemini api but I ran out of credits so I resorted to using an open source package named “g4f” which lets you access LLM APIs for free
- The Tavily API returned links to the most relevant articles along with the content in them, but the content was not always the entire text in the webpage. I tried extracting the text from the webpages using beautiful soup or selenium, but due to the variety in the web page structures, I resorted to using the diffbot API to extract the text content from a web page.
- I compared your linkedin and the documents I was given and I found no mention of you working at Stardust on your linkedin, I wasn't sure if I had to take the documents as the primary source of information, so I did. But if this mismatch was intentional and I had to deal with both sources of data (if the data you gave me was more extensive but the data in your linkedin was accurate), here's how I would solve the problem in a generalized way, not restricting the solution to just the Stardust part:
 - I would make a database consisting of the info from your linkedin, add an extra node in the beginning of the architectures and run a quick search in the database to check if I can find any matching instances of the data required in the linkedin database, if yes I'd proceed with the previous architectures.
- I was unable to return relevant documents for some queries mentioned in the final templates, the list below mentions the queries and the reasons due to which I was unable to return answers for those:
 - Performance evaluations: they seemed like documents internal to the company
 - Objectives and scopes of some projects without specific names but specific technologies like those mentioned in the stardust (it was mentioned that you led teams like the Platform teams and the API teams). I was unable to find any specific info on them, so I asked Gemini to return a general answer in such cases.
 - Measurable outcomes for the projects led as this seems like information internal to the company and it wasn't listed anywhere in the data provided or in your linkedin.
 - Letters of recommendation: I looked through your LinkedIn to see if I could find any data for this like specific endorsements from colleagues but I could only find skill endorsements, hence I couldn't return anything for this

- Patents and publications: I can just manually add the links to them
- Awards and recognition and media coverage about you: I did a google search but couldn't find any info
- Proof of membership for organizations that grant fellowship: seemed like private info that I couldn't get hold of. Looked through your linkedin too to see if I could find any posts about fellowships, but couldn't find any.
- Information on the selection committee for memberships and Association Events and Contributions
- I noticed that the section in the first question which was related to awards and recognitions received by the company had a few hallucinations when the query was related to Ava Labs, Stardust and Matterport.

Overall, I had fun working on this assignment and the learning was invaluable.